Design Analysis Report: Digital Payment Gateway Simulation

Name: Ashutosh Tiwari | **Branch:** CSE, Section B, Semester V | **University Reg. No.:** 202401100200098 | **University Roll No.:** 2400290100098 | KIET Deemed to be University, Ghaziabad

1. Introduction

A digital payment gateway is the software layer that sits between a customer, a merchant and a bank. It receives payment details, checks who the user is, decides whether the payment looks safe, asks the bank to move the money and then reports the result. This project simulates that whole journey in software so that the design can be studied without handling real money.

This report analyses the design of the Digital Payment Gateway Simulation. It explains what the system must do, how the parts are organised, how the parts talk to each other, and how good the resulting design is in terms of cohesion, coupling, security, failure handling and future growth. The analysis is based on the UML and data flow models prepared for this project: the Component Diagram, State Transition Diagram, Collaboration Diagram, Sequence Diagram, Activity Diagram and Data Flow Diagram.

2. Objectives and Scope

Objectives

1. Simulate the complete life cycle of an online payment, from order creation to receipt.
2. Support more than one payment method (card, UPI, net banking and wallet).
3. Keep authentication, fraud checking, bank communication and record keeping as separate, replaceable parts.
4. Make every transaction traceable by storing its final status.

In scope: order intake, user authentication (OTP/PIN), fraud and risk check, simulated bank debit, transaction storage, notification, and refunds of successful payments.

Out of scope: real bank integration, real money movement, PCI-DSS certification, chargeback disputes, and multi-currency conversion.

3. Requirements Analysis

3.1 Functional requirements

ID	Requirement
FR1	The merchant application shall create an order with a unique order ID and amount.
FR2	The customer shall be able to choose a payment method and enter payment details.
FR3	The system shall authenticate the user using OTP or PIN before any money is moved.
FR4	The system shall run a fraud and risk check (amount limit, repeated attempts) on every payment.
FR5	The system shall send an approved payment to the bank simulator and receive an approve or decline response.
FR6	The system shall store every transaction with its status (SUCCESS, FAILED, DECLINED, CANCELLED, REFUNDED).
FR7	The system shall notify the customer and the merchant of the result by SMS or e-mail.
FR8	The system shall allow a refund request for a successful payment within the refund window.

3.2 Non-functional requirements

Quality	Requirement
Security	Credentials and card data must not be stored in plain text or written to logs.
Reliability	A bank time-out must end in a clear FAILED state, never in an undefined state.

Quality	Requirement
Performance	A normal payment should complete within a few seconds in the simulation.
Maintainability	Each service should be changeable without rewriting the others.
Extensibility	A new payment method should be added without changing the core transaction logic.
Auditability	Every status change should be traceable through the transaction record.

4. Class Identification

Classes were identified by taking the nouns in the requirements and removing duplicates and attributes. They were then classified as **strong classes** (they can exist on their own and have their own identity) and **weak classes** (their existence depends on another class).

Class	Type	Main attributes	Main operations	Reason for classification
Customer	Strong	customerId, name, email, phone	enterPaymentDetails(), viewReceipt()	Exists independently of any payment.
Merchant	Strong	merchantId, name, apiKey	createOrder(), receiveStatus()	Exists independently of any order.
Order	Strong	orderId, amount, currency, createdAt	validate()	Has its own identity and life cycle.
Transaction	Strong	transactionId, status, amount, timestamp	updateStatus()	The central record of the system.

Class	Type	Main attributes	Main operations	Reason for classification
PaymentGateway	Strong	gatewayId, config	processPayment(), refund()	Controller that coordinates the payment.
BankAccount	Strong	accountNo, balance	debit(), credit()	Exists independently inside the bank simulator.
PaymentMethod	Strong (abstract)	methodId, type	pay()	Parent of Card, UPI, NetBanking and Wallet.
OTP	Weak	code, expiry, attempts	verify()	Cannot exist without a Customer and a Transaction.
Receipt	Weak	receiptNo, issuedAt	generate()	Cannot exist without a Transaction.
Notification	Weak	message, channel, sentAt	send()	Exists only because of a Transaction event.
Refund	Weak	refundId, amount, reason	process()	Cannot exist without a successful Transaction.

Key relationships

- A Customer places many Orders; each Order belongs to exactly one Merchant.
- An Order has one or more Transaction attempts (a failed attempt can be retried).
- A Transaction uses exactly one PaymentMethod and is linked to one BankAccount debit.

- A Transaction owns its Receipt, OTP, Notification and Refund records (composition), which is why these are weak classes.
- Card, UPI, NetBanking and Wallet inherit from PaymentMethod (generalisation).

5. Design Overview

5.1 Architectural style

The design follows a **layered, service-oriented structure**. A single entry point, the Payment Gateway API, receives requests from the merchant and delegates the work to specialised services. The Transaction Manager acts as the coordinator that controls the order of the steps.

Component	Responsibility
Customer Client App	Collects payment details and shows the result to the customer.
Merchant Checkout Application	Creates the order and talks to the gateway.
Payment Gateway API	Single entry point; validates and routes requests.
Authentication Service	Verifies credentials and OTP/PIN.
Transaction Manager	Controls the payment life cycle and calls the other services in order.
Fraud Detection Service	Applies risk rules before approval.
Payment Method Processor	Holds the logic for card, UPI, net banking and wallet payments.
Bank / Acquirer Simulator	Plays the role of the bank (balance check, debit, approve or decline).
Transaction Database	Stores transaction data and receipts.
Notification Service	Sends SMS or e-mail confirmations.

5.2 Payment flow in one paragraph

The customer enters details in the client app, which passes the order to the merchant. The merchant asks the gateway to create a payment. The gateway authenticates the user, hands the payment to the Transaction Manager, which asks the Fraud Detection Service for a risk level. If the risk is low, the Payment Method Processor asks the bank simulator to debit the account. The Transaction Manager stores the result, triggers the notification, and the gateway returns the final status to the merchant, which shows it to the customer.

6. Summary of the Design Models

Each model looks at the same system from a different angle. Together they check that the design is complete and consistent.

Model	View it gives	Main finding for the design
Component Diagram	Structure: which parts exist and which interfaces connect them	Ten components with clear interfaces such as IPaymentAPI, IAuthenticate, IRiskCheck and IBankGateway; the Transaction Manager is the hub.
State Transition Diagram	Life cycle of one transaction	Nine states from Initiated to Refunded; every failure path ends in a Failed or Cancelled state, so no transaction is left undefined.
Collaboration Diagram	Which objects cooperate and in what numbered order	Thirteen messages show that the Payment Gateway and Transaction Manager are the busiest objects.
Sequence Diagram	Time order of the messages, with success and decline branches	Fraud check happens before the bank is contacted, so risky payments never reach the bank.

Model	View it gives	Main finding for the design
Activity Diagram	Workflow and responsibilities across four swimlanes	Three decision points (auth valid, risk low, bank approves) all lead to a common failure branch; saving the record and sending the notification can run in parallel.
Data Flow Diagram	How data moves between entities, processes and stores	Three external entities, six processes and three data stores (User Accounts, Fraud Rules, Transactions); failed attempts are recorded as well as successful ones.

7. Design Analysis

7.1 Cohesion

Each component does one job: authentication only authenticates, the fraud service only scores risk, the notification service only sends messages. This is **high cohesion**, which makes each part easy to understand, test and replace. The one component with a wider role is the Transaction Manager, but its role is a single, well-defined one: controlling the order of the steps.

7.2 Coupling

Components talk to each other only through named interfaces (for example IRiskCheck and IBankGateway) and not through each other's internals. This gives **low coupling**: the bank simulator can be replaced by a different bank implementation without touching the Transaction Manager, as long as the IBankGateway interface is respected. The main coupling risk is that the Transaction Manager depends on five other components, so a change in its flow affects the whole payment path.

7.3 Design principles applied

Principle or pattern	How it appears in the design
Single Responsibility	Each service has one reason to change.
Open/Closed	New payment methods are added by extending PaymentMethod, not by editing the Transaction Manager.
Dependency Inversion	The Transaction Manager depends on interfaces such as IBankGateway, not on a concrete bank.
Facade	The Payment Gateway API hides the internal services behind one entry point.
Strategy	Card, UPI, net banking and wallet are interchangeable ways of paying.
State	A transaction moves through defined states with allowed transitions only.
Observer (optional)	The Notification Service can listen for transaction status changes instead of being called directly.

7.4 Security analysis

Risk	Design response
Unauthorised user	Authentication (OTP/PIN) runs before any payment processing; three wrong attempts lead to Failed.
Fraudulent payment	Risk check runs before the bank is contacted; flagged payments are declined.
Data exposure	Sensitive fields are kept out of logs and the transaction record stores only masked card data.
Replay or tampering	Each order has a unique ID and a payment can succeed only once.
Unauthorised merchant	The merchant API key is checked at the gateway entry point.

7.5 Failure handling and reliability

The State Transition Diagram shows that every unsuccessful path (invalid authentication, fraud flagged, bank declined or time-out) ends in the Failed state and is recorded. A bank time-out is treated as a decline so the customer is never charged without a recorded result. A remaining risk is a time-out that happens after the bank has actually debited the account. A real system would need a reconciliation step for this case; in this simulation it is listed as a limitation.

7.6 Performance and scalability

Because the services are separate, the busiest ones (fraud check and bank communication) can be scaled on their own. The Transaction Database is the shared resource and is the most likely bottleneck. Saving the transaction and sending the notification run in parallel (shown by the fork in the Activity Diagram), which shortens the response time seen by the customer.

7.7 Extensibility

A new payment method only needs a new subclass of PaymentMethod and an entry in the Payment Method Processor. A new fraud rule only needs a new entry in the Fraud Rules store. A new notification channel (for example push notification) only needs a change in the Notification Service. None of these changes touch the other components.

8. Design Alternatives and Decisions

Decision	Option chosen	Alternative considered	Why this choice
Overall structure	Separate services behind one gateway API	One single large program	Easier to maintain and to show each responsibility in the diagrams.
Order of checks	Authenticate, then fraud check, then bank	Fraud check first	Authentication is cheaper and removes invalid users early; the expensive bank call happens last.

Decision	Option chosen	Alternative considered	Why this choice
Bank communication	Through an interface (IBankGateway)	Direct calls to one bank class	Allows replacing or adding banks without changing core logic.
Notification	Triggered by the Transaction Manager	Separate scheduled job	Customer gets an immediate result.
Failure records	Failed attempts are stored too	Store only successful payments	Needed for audit, dispute handling and fraud analysis.

9. Strengths and Limitations

Strengths

- Clear separation of responsibilities, with high cohesion and low coupling.
- Complete life cycle coverage, including failure, cancel and refund paths.
- Consistent models: the same components, states and messages appear in all six diagrams.
- Easy to extend with new payment methods, rules and notification channels.

Limitations

- The Transaction Manager is a central point that many components depend on.
- The Transaction Database is a single shared store and a possible bottleneck.
- Bank time-out after a successful debit is not fully handled (no reconciliation).
- Only the main refund flow is modelled; partial refunds and chargebacks are not.
- Security is described at design level only; encryption and key management are not simulated.

10. Conclusion

The Digital Payment Gateway Simulation has a clean, service-based design in which each component has one responsibility and communicates through defined interfaces. The six models agree with each other and together cover structure, behaviour, interaction, workflow and data movement. The design is easy to maintain and extend, and its failure

paths are explicit. The main areas to improve are reducing the load on the Transaction Manager, adding reconciliation for bank time-outs, and extending the refund and security parts of the design. With these improvements the design could be taken from a classroom simulation toward a realistic payment system.